

FASE 3: Proyecto Final

MIPS 32-BIT: MERGESORT

Implementación de hardware segmentado (Pipeline)

EQUIPO:

- Luis Giovanni Rodríguez Hernández
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encina

EL PIPELINE DE 5 ETAPAS



IF & ID

Búsqueda y Decodificación: El PC localiza la instrucción en MEMI (formato Big-Endian) y UNITCTRL genera las señales de control.



EX (EJECUCIÓN)

Cálculo Crítico: La ALU realiza operaciones matemáticas y lógicas necesarias para comparar elementos del Mergesort.



MEM & WB

Memoria y Escritura: Acceso a MEMA para guardar variables temporales del arreglo y guardado final en el Banco de Registros.



Sincronización: Los buffers IF/ID, ID/EX, EX/MEM y MEM/WB aseguran que los datos no colisionen entre ciclos de reloj.

| UNIDAD LÓGICA ARITMÉTICA

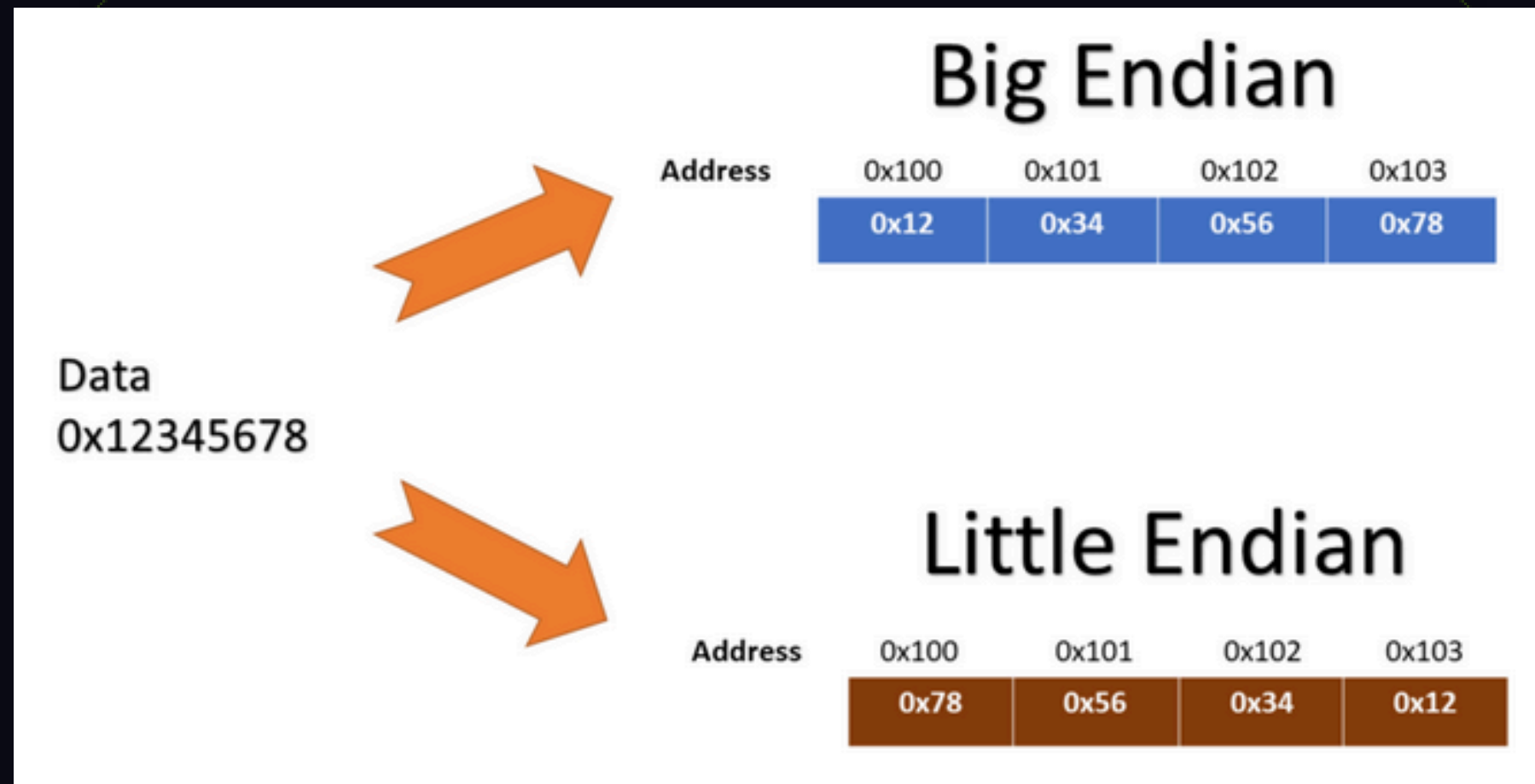
ALUMIPS & ALUCTRL

El motor de comparación para el Mergesort.

- ✓ **Comparación con Signo:** Vital para Mergesort al comparar valores de arreglos mediante ``$signed(A) < $signed(B)``.
- ✓ **Lógica Predictiva:** ALUCTRL traduce el *Opcode* y *Func* para activar la operación exacta en nanosegundos.
- ✓ **Cálculo de Direcciones:** Realiza sumas de punteros de pila (Stack Pointer) para guardar el contexto recursivo.



MEMORIA DE DATOS E INSTRUCCIONES



MEMI (INSTRUCCIONES)

Implementa **Big-Endianness** cargando 4 bytes simultáneos para formar una instrucción de 32 bits desde *inst.txt*.

MEMA (DATOS)

Actúa como la "bodega" donde el Mergesort guarda los sub-arreglos divididos antes de la fase de mezcla (Merge).

REGISTRO DE DATOS: EL BANCO (BR)

32

REGISTROS DISPONIBLES

ACCESO ULTRA-RÁPIDO

El módulo **BR** es la memoria de trabajo más cercana al procesador. En

Mergesort, se utiliza para:

- ⚡ Guardar punteros de inicio (\$p\$), medio (\$q\$) y fin (\$r\$).
- ⚡ Almacenar temporalmente los valores de comparación.
- ⚡ Lectura asíncrona de dos puertos para operaciones de suma inmediatas.

EL ALGORITMO MERGESORT

¿Cómo lo procesa nuestro hardware?

DESAFÍOS EN HARDWARE



RECURSIVIDAD

Requiere saltos condicionales (``beq``) y absolutos (``j``). El hardware usa **JumpTarget** para saltar a funciones de división.



GESTIÓN DE PILA

Para cada llamada recursiva, ``sw`` (Store Word) guarda la dirección de retorno en la memoria para no perder el flujo.



FASE DE MEZCLA

Utiliza la operación ``slt`` (Set on Less Than) de la ALU para decidir qué elemento es menor y colocarlo en orden.

MATRIZ DE CONTROL (UNITCTRL)

Instrucción	RegDst	AluSrc	MemToReg	RegWrite	Branch	Jump
R-Type (add, slt)	1	0	0	1	0	0
LW (Carga)	0	1	1	1	0	0
SW (Guarda)	X	1	X	0	0	0
BEQ (Rama)	X	0	X	0	1	0
J (Salto)	X	X	X	0	0	1

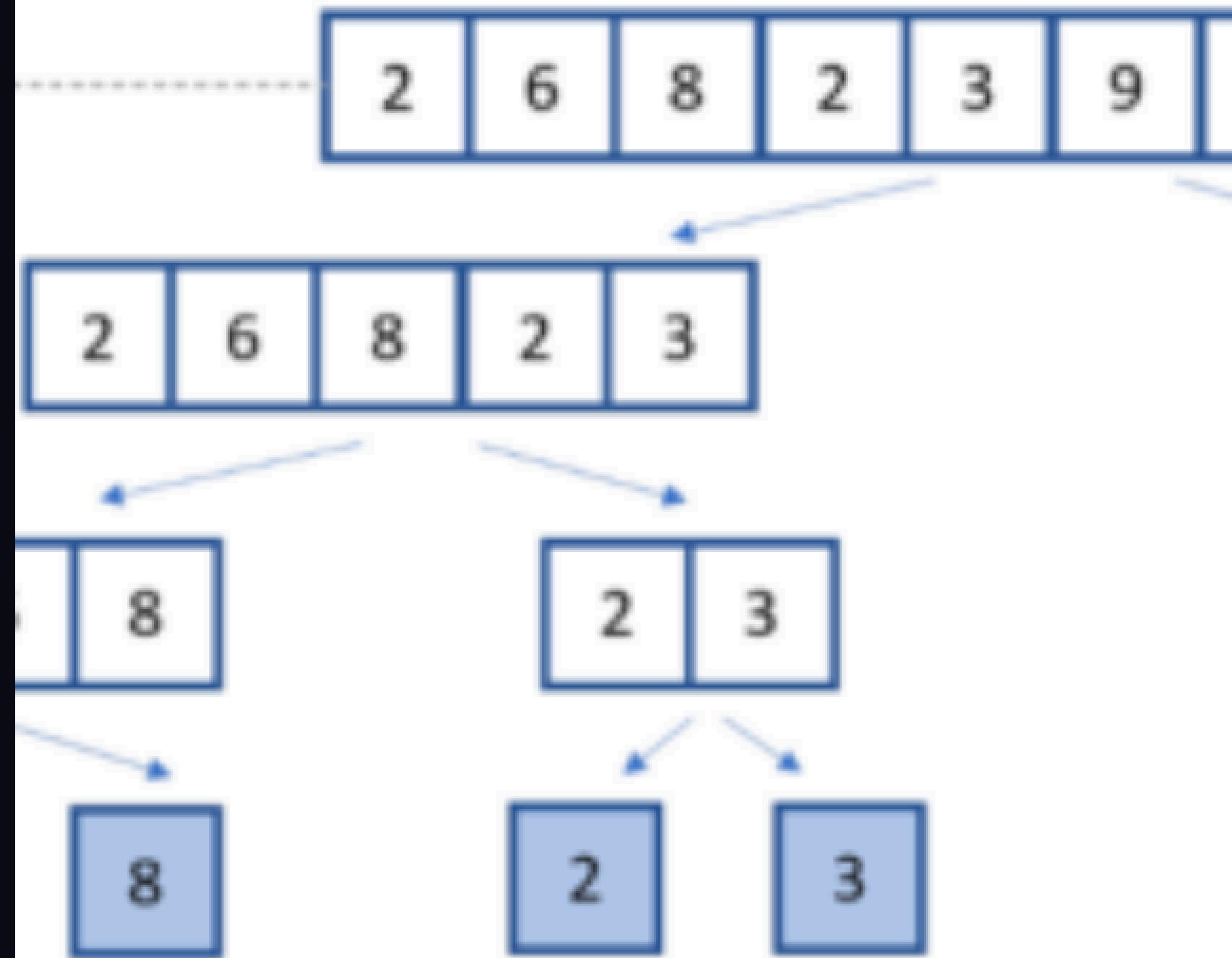
* X denota valores irrelevantes (Don't Care) para esa operación específica.

LÓGICA DE MEZCLA

Ordenamiento bit a bit.

Durante la ejecución, el procesador divide el problema en tareas atómicas. La ALU resuelve comparaciones constantes mientras el **Datapath** redirige los resultados a la memoria.

Gracias al **SignExtend**, el procesador maneja correctamente números negativos en el arreglo, asegurando que el ordenamiento sea matemáticamente preciso.



| VALIDACIÓN: TB_DPTR

SIMULACIÓN EN TIEMPO REAL

Nuestro Testbench (`TB\_DPTR`) simula el latido del procesador cada 20ns. Monitorea:

- >_ **PC (IF):** La dirección de la instrucción actual.
- >_ **Instr (ID):** La instrucción binaria cargada.
- >_ **ALUResult:** El resultado del cálculo en la etapa EX.
- >_ **AW (WB):** El registro final donde se guarda la información.



CONCLUSIONES DEL PROYECTO

MODULARIDAD

El diseño separado por etapas permite identificar fallos rápidamente y optimizar cada bloque (ALU, MEM, BR) de forma independiente.

ESCALABILIDAD

Aunque implementamos Mergesort, el set de 28 instrucciones permite ejecutar cualquier algoritmo lógico de complejidad similar.

RENDIMIENTO

La arquitectura segmentada (Pipeline) triplica la eficiencia teórica comparada con un diseño de ciclo único al procesar múltiples fases a la vez.